

Laboratorio 2: Operación «Claves de Guerra»

Criptografía aplicada

Universidad Técnica Federico Santa María
Departamento de Informática
Redes de Computadores - 2026-1

Grupo X

Moises Gonzalez
Elena Vargas
Dimitri Volkov
Sofia Reyes

1. Introducción

Este informe documenta la implementación del laboratorio 2 del ramo Redes de Computadores. El trabajo está dividido en cuatro fases: (1) generar identidades criptográficas a partir del nombre y rol de cada miembro, (2) cifrar un archivo grande usando cifrado híbrido (AES + RSA) y comparar los modos ECB y CBC, (3) implementar firma digital con el protocolo Sign-then-Encrypt, y (4) repartir una clave secreta de 256 bits con el esquema de Shamir.

El código está dividido en cuatro archivos (fase1.py, fase2.py, fase3.py y fase4.py) y se ejecuta en orden con make run. La única dependencia externa es la librería *cryptography* de Python.

2. Desarrollo por fase

2.1. Fase 1 — Identidad criptográfica (KDF + RSA)

Cada miembro del consejo (A, B, C y D) genera su identidad a partir de su Nombre y Rol. El proceso es:

1. **Generar un SALT aleatorio** de 16 bytes con `os.urandom`.
2. **Aplicar PBKDF2-HMAC-SHA256** sobre la cadena `nombre|rol` junto con el SALT, pidiendo 32 bytes de salida y 200.000 iteraciones. El resultado es una clave de 256 bits que llamamos *clave KDF*.
3. **Generar un par RSA de 2048 bits**.
4. **Guardar en disco:** el SALT en un archivo `.salt`, la llave pública en PEM, y la llave privada en PEM cifrada usando la clave KDF como contraseña.

El SALT es importante porque, si dos personas tuvieran el mismo Nombre + Rol, sin SALT obtendrían la misma clave KDF. Con SALT, cada persona obtiene una clave única aunque sus datos coincidan. Las 200.000 iteraciones hacen que probar muchas combinaciones por fuerza bruta sea lento.

La llave privada queda cifrada con la clave KDF, así que solo quien conozca el Nombre, Rol y SALT puede abrirla.

2.2. Fase 2 — Cifrado híbrido y comparación ECB vs CBC

El plano del silo (786.486 bytes) es muy grande para cifrarlo directamente con RSA, así que se usa **cifrado híbrido**:

1. Se genera una **clave AES aleatoria** de 16 bytes y un **IV aleatorio**.
2. Se cifra el archivo con **AES-128-CBC** y padding PKCS7.
3. Se cifra la clave AES con **RSA-OAEP** usando la pública del destinatario.
4. Se empaqueta todo en un archivo: `[len(enc_clave)] [enc_clave] [iv] [ciphertext]`.

El receptor descifra primero la clave AES con su privada y luego descifra el archivo. El test del script verifica que el archivo recuperado es idéntico al original.

ECB vs CBC. En modo **ECB**, cada bloque de 16 bytes se cifra de forma independiente. Si dos bloques del archivo plano son iguales, sus bloques cifrados también lo son. En una imagen, las zonas de color uniforme producen muchos bloques iguales, y en el archivo cifrado se siguen distinguiendo los contornos del original.

En modo **CBC**, antes de cifrar cada bloque se hace un XOR con el bloque cifrado anterior (el primero usa un IV aleatorio). Esto rompe los patrones: aunque dos bloques del original sean iguales, sus bloques cifrados quedan distintos. El resultado se ve como ruido aleatorio. Por eso elegimos CBC y no ECB. La sección 4 muestra la comparación visual.

2.3. Fase 3 — Firma digital y Sign-then-Encrypt

Para que el receptor pueda confirmar quién envió un mensaje, el emisor lo firma con su llave privada. El protocolo es **firmar primero, cifrar después** (Sign-then-Encrypt):

1. **Firmar el mensaje** con RSA-PSS y SHA-256 usando la privada del emisor. La firma ocupa 256 bytes (RSA-2048).
2. **Concatenar** mensaje + firma.
3. **Cifrar el paquete** con cifrado híbrido (AES-128-CBC + RSA-OAEP de la pública del receptor), igual que en la fase 2.

El receptor hace el proceso inverso (Decrypt-then-Verify): descifra, separa los últimos 256 bytes (la firma) del resto (el mensaje), y verifica la firma con la pública del emisor esperado. Si la firma no valida, el script imprime SABOTAJE DETECTADO.

El script prueba cuatro casos: (1) un mensaje legítimo de A a B, (2) otro de B a C, (3) un mensaje al que se le altera 1 bit en tránsito, y (4) un intento de C de hacerse pasar por A. Los casos 1 y 2 verifican correctamente; los casos 3 y 4 disparan la alerta.

2.4. Fase 4 — Esquema de Shamir (3 de 4)

La clave maestra de 256 bits se reparte entre los 4 miembros del consejo, pero solo 3 cualquiera pueden reconstruirla. El esquema de Shamir funciona así:

1. Se construye un polinomio aleatorio $f(x) = s + a_1 \cdot x + a_2 \cdot x^2$, donde s es el secreto y a_1, a_2 son coeficientes aleatorios.
2. Cada miembro i recibe el punto $(i, f(i))$ con $i = 1, 2, 3, 4$.
3. Para reconstruir el secreto, se necesita el valor de $f(0)$. Con 3 puntos cualquiera, la interpolación de Lagrange permite calcularlo de forma exacta.
4. Con menos de 3 puntos, hay infinitos polinomios distintos que pasan por ellos, así que el secreto no se puede determinar.

Toda la matemática se hace módulo un primo grande P (mayor que 2^{256}) y con enteros de Python. **No usamos punto flotante:** las divisiones de Lagrange se calculan como multiplicación por el inverso modular, usando `pow(a, -1, P)`. La razón se explica en la sección 3.C.

El script prueba cinco casos: con 3 fragmentos (A,B,C), con otros 3 (B,C,D), con los 4, con 2 fragmentos y con 1. Solo los tres primeros reconstruyen el secreto correctamente; en los otros dos, el resultado de Lagrange es un valor que no tiene relación con el secreto real.

3. Respuestas al interrogatorio

A) El orden de los factores: ¿por qué firmar antes de cifrar?

Si firmamos primero y ciframos después, la firma queda dentro del ciphertext. Esto tiene tres ventajas concretas:

1. **El emisor queda oculto.** Si alguien intercepta el paquete cifrado, no puede saber quién lo firmó porque la firma está dentro del cifrado y necesita la llave privada del receptor para abrirla. En cambio, si ciframos primero y firmamos después, la firma viaja en claro y revela al emisor.
2. **Evita ataques de reenvío de firma.** Si la firma fuera sobre el ciphertext, un atacante podría descartar la firma original y agregar la suya, haciendo creer al receptor que el mensaje viene de él, sin haberlo descifrado. Firmando el plano, la firma queda atada al contenido real y no puede sustituirse sin descifrar primero.
3. **Lo que se firma es lo que se envió.** El emisor firma el mensaje original (lo que él escribió), no un objeto dependiente de claves de terceros. Si después se discute «¿qué dijo realmente?», basta verificar la firma sobre el mensaje plano.

B) La entropía de los nombres: ¿cómo garantiza el SALT que no se puedan precalcular llaves?

Sin SALT, si dos miembros tienen el mismo Nombre y Rol, PBKDF2 produce exactamente la misma clave. Peor aún: un atacante podría construir una vez una **tabla precalculada** con muchos pares (Nombre, Rol) y sus claves derivadas, y reutilizarla contra cualquier víctima. Como Nombre y Rol son datos de baja entropía (hay nombres comunes y roles típicos), esa tabla sería relativamente barata de construir.

El SALT mete 16 bytes (128 bits) aleatorios dentro de la entrada de PBKDF2. El efecto es:

1. **Cada miembro tiene SALT distinto**, así que dos miembros con Nombre y Rol idénticos generan claves completamente diferentes.
2. **La tabla precalculada deja de servir:** el atacante tendría que construir una tabla nueva por cada SALT posible, y hay 2^{128} SALTs distintos, lo que es inviable.
3. **El atacante se ve forzado a atacar a cada víctima por separado**, y por las 200.000 iteraciones de PBKDF2, cada intento individual ya es lento.

C) El error del punto flotante: ¿por qué la aritmética decimal rompe el esquema de Shamir?

El esquema de Shamir requiere divisiones exactas (los términos de Lagrange tienen la forma $1/(x_i - x_j)$). Si esas divisiones se hacen con float (punto flotante), aparecen dos problemas:

1. **Pérdida de precisión.** Los double de Python tienen 53 bits de precisión. Nuestros y_i son enteros de cientos de bits, mucho más grandes que lo que cabe en un double. Al hacer las divisiones, el resultado se redondea silenciosamente y deja de ser exacto. Cuando sumamos los términos de Lagrange, el secreto reconstruido **no coincide** con el original.
2. **Cancelación al restar.** Las fórmulas de Lagrange tienen sumas con signos alternados de números muy parecidos en magnitud. En punto flotante, esas restas pierden la mayoría de los dígitos significativos y el error se amplifica.

Por eso usamos **aritmética modular sobre enteros**. Trabajando módulo un primo P , todo elemento no nulo tiene un inverso multiplicativo exacto, y Python soporta enteros de tamaño arbitrario sin perder precisión. La división a/b se calcula como $a \cdot b^{-1} \bmod P$, y nunca aparece un redondeo.

4. Evidencias de ejecución

Las siguientes capturas muestran la salida real de cada fase al ejecutar make run.

```
fase1.py

=====
FASE 1: Identidad criptográfica (KDF + RSA)
=====

Generando identidad para MiembroA (Moises Gonzalez, Comandante)
SALT      : 16bf7e1d2b4280dbb7247eea13fec543
Clave KDF  : f6da65149f686bc3... (32 bytes)
Llaves RSA : MiembroA_private.pem, MiembroA_public.pem

Generando identidad para MiembroB (Elena Vargas, Estratega)
SALT      : 95e058fb5d76df825e07e8ec19647316
Clave KDF  : 282b15c7db63c623... (32 bytes)
Llaves RSA : MiembroB_private.pem, MiembroB_public.pem

Generando identidad para MiembroC (Dimitri Volkov, Ingeniero)
SALT      : 1ad8208e1dc929472150683fbf7ed69c
Clave KDF  : 9d7cbca395a05da6... (32 bytes)
Llaves RSA : MiembroC_private.pem, MiembroC_public.pem

Generando identidad para MiembroD (Sofia Reyes, Operador)
SALT      : e277832df67978c919e01536b0c4be96
Clave KDF  : 158e11645c008321... (32 bytes)
Llaves RSA : MiembroD_private.pem, MiembroD_public.pem

Fase 1 completada. Archivos en ./keys/
```

Figura 1. Fase 1: generación de identidades.

```
fase2.py

=====
FASE 2: Cifrado híbrido y prueba ECB vs CBC
=====

[1] Cifrado híbrido del plano del silo (A -> B)
Archivo cifrado: output/silo_hybrid_for_B.bin (786770 bytes)
Clave AES (hex): 1dd70658b9efb7b30a174463dc207d56
IV (hex)       : d1d28622fcc4e337ff7377b57812d2b4
Archivo recuperado: output/silo_recuperado.bmp (786486 bytes)
Verificación: original == recuperado -> True

[2] Prueba visual ECB vs CBC

Comparación ECB vs CBC sobre data/silo_circuito.bmp
output/silo_ECB.bmp -> patrones del original aún visibles
output/silo_CBC.bmp -> aspecto de ruido aleatorio

Fase 2 completada. Archivos en ./output/
```

Figura 2. Fase 2: cifrado híbrido del plano del silo y verificación de descifrado.

```
fase3.py

=====
FASE 3: Firma digital y Sign-then-Encrypt
=====

[Caso 1] Mensaje legítimo A -> B
Mensaje original: Activar protocolo Golgotha. Coordenadas: 41N 70W. Hora: 03:00 UTC.
MiembroA -> MiembroB
mensaje : 66 bytes
firma   : 256 bytes
paquete : 610 bytes (enc_clave + iv + cifrado)
MiembroB <- MiembroA: firma válida ✓
Mensaje descifrado: Activar protocolo Golgotha. Coordenadas: 41N 70W. Hora: 03:00 UTC.

[Caso 2] Mensaje legítimo B -> C
MiembroB -> MiembroC
mensaje : 49 bytes
firma   : 256 bytes
paquete : 594 bytes (enc_clave + iv + cifrado)
MiembroC <- MiembroB: firma válida ✓

[Caso 3] Alguien altera 1 bit del paquete (intento de sabotaje)
Bit alterado en byte 305 de 610
>>> SABOTAJE DETECTADO (firma inválida)

[Caso 4] C intenta firmar diciendo ser A
MiembroC -> MiembroB
mensaje : 38 bytes
firma   : 256 bytes
paquete : 578 bytes (enc_clave + iv + cifrado)
>>> SABOTAJE DETECTADO (firma inválida)

Fase 3 completada.
```

Figura 3. Fase 3: firma digital. Mensajes legítimos (casos 1 y 2) y dos ataques (casos 3 y 4) detectados.

```
fase4.py

=====
FASE 4: Esquema de Shamir (t=3, n=4)
=====

Clave maestra (hex): a9856283bb68b412c3ca333e0f37ff586e5742430872ed7e9f6b5a4f4cc56327

Fragmentos generados:
Miembro 1: 0x1cb1ef436e518eec0e5e27dd2e47bbf53314...
Miembro 2: 0xf544021018d389b2b02538c83ad45d408416...
Miembro 3: 0x17e6f298b9b2fd0d55ec830e00309d9c7f85...
Miembro 4: 0x166a06aa96c2dc428f1cb661a3d1c34e98e1...
Guardados en output/share_1.txt ... share_4.txt

Casos de reconstrucción:

Caso A: 3 fragmentos (A, B, C) (k=3)
Fragmentos usados: [1, 2, 3]
Reconstruido: 0xa9856283bb68b412c3ca333e0f37ff58...
>>> COINCIDE con el secreto: ÉXITO

Caso B: 3 fragmentos (B, C, D) (k=3)
Fragmentos usados: [2, 3, 4]
Reconstruido: 0xa9856283bb68b412c3ca333e0f37ff58...
>>> COINCIDE con el secreto: ÉXITO

Caso C: 4 fragmentos (todos) (k=4)
Fragmentos usados: [1, 2, 3, 4]
Reconstruido: 0xa9856283bb68b412c3ca333e0f37ff58...
>>> COINCIDE con el secreto: ÉXITO

Caso D: 2 fragmentos (insuficiente) (k=2)
Fragmentos usados: [1, 2]
Reconstruido: 0xa0f9e65db15e53cf1b9fc2dd8e232165...
Secreto real: 0xa9856283bb68b412c3ca333e0f37ff58...
>>> NO COINCIDE: el resultado parece aleatorio

Caso E: 1 fragmento (insuficiente) (k=1)
Fragmentos usados: [1]
Reconstruido: 0x1cb1ef436e518eec0e5e27dd2e47bbf5...
Secreto real: 0xa9856283bb68b412c3ca333e0f37ff58...
>>> NO COINCIDE: el resultado parece aleatorio

Fase 4 completada.
```

Figura 4. Fase 4: Shamir. Con 3 o más fragmentos se reconstruye el secreto; con menos, el resultado no coincide.

```
ls keys/ output/

$ ls -l keys/ output/
keys/:
MiembroA.salt
MiembroA_private.pem
MiembroA_public.pem
MiembroB.salt
MiembroB_private.pem
MiembroB_public.pem
MiembroC.salt
MiembroC_private.pem
MiembroC_public.pem
MiembroD.salt
MiembroD_private.pem
MiembroD_public.pem

output/:
img
msg_A_to_B.bin
msg_A_to_B_tampered.bin
msg_B_to_C.bin
share_1.txt
share_2.txt
share_3.txt
share_4.txt
silo_CBC.bmp
silo_ECB.bmp
silo_hybrid_for_B.bin
silo_recuperado.bmp
```

Figura 5. Archivos generados en keys/ y output/.

4.1. Comparación visual ECB vs CBC

La fase 2 cifra el BMP del silo en ambos modos con la misma clave AES y guarda el resultado como un archivo BMP visible (manteniendo el header BMP de 54 bytes intacto y cifrando solo el cuerpo).

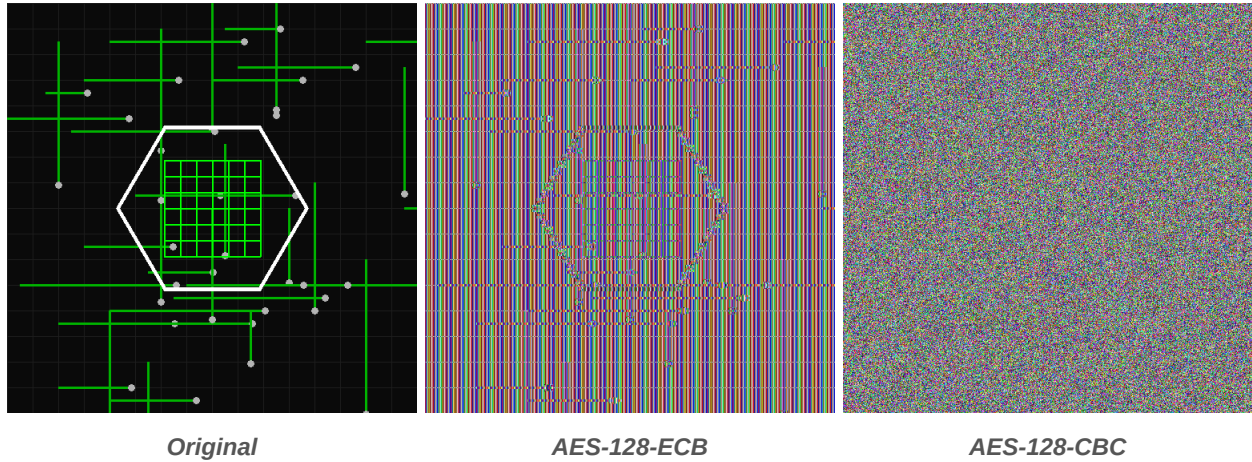


Figura 6. Comparación visual de los modos de cifrado.

Análisis. En la imagen cifrada con ECB todavía se distinguen claramente la silueta del hexágono y el patrón de cuadrícula del centro: las regiones de color uniforme del original producen bloques cifrados idénticos, así que la estructura macro de la imagen sobrevive al cifrado. En cambio, la imagen CBC se ve como ruido aleatorio sin ninguna estructura reconocible, porque cada bloque depende del cifrado del anterior. Esta es la razón por la que **elegimos CBC** y no ECB en la fase 2 (cifrado híbrido) y en la fase 3 (Sign-then-Encrypt).

5. Conclusiones

Las cuatro fases del laboratorio quedaron implementadas y funcionan como se espera:

- La fase 1 genera identidades reproducibles a partir del Nombre y Rol, protegidas con SALT y PBKDF2.
- La fase 2 cifra archivos grandes de forma eficiente combinando AES y RSA, y deja claro por qué CBC es preferible a ECB.
- La fase 3 garantiza que solo el destinatario puede leer el mensaje y que la integridad y autoría se verifican: cualquier alteración (incluso de un solo bit) o intento de suplantación se detecta.
- La fase 4 reparte un secreto de 256 bits entre 4 miembros y cumple la propiedad de umbral: con 3 o más fragmentos se reconstruye, con menos no.

El uso de aritmética entera modular en Shamir resultó esencial: intentar las mismas cuentas en punto flotante introduce errores de redondeo que arruinan la reconstrucción.

Posibles mejoras: usar AES-GCM en lugar de AES-CBC para tener autenticación e integridad incluidas en el modo simétrico, e incluir el identificador del destinatario dentro del mensaje firmado para evitar reenvíos.